



HTTP, Website Architecture and Middleware

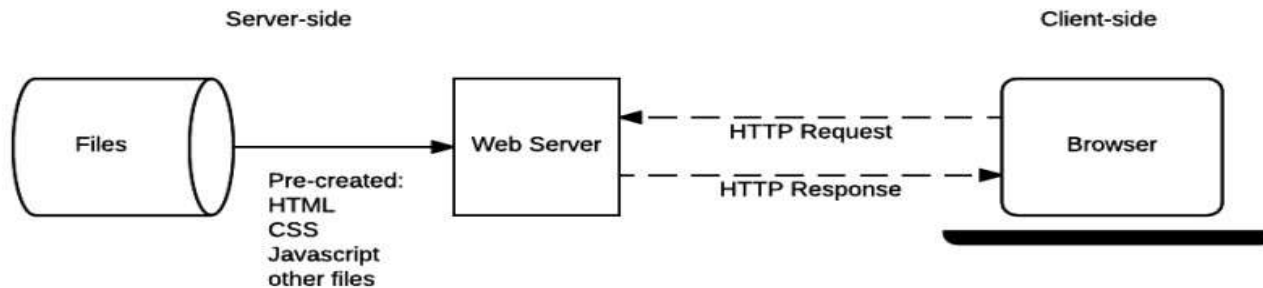


Overview of HTTP

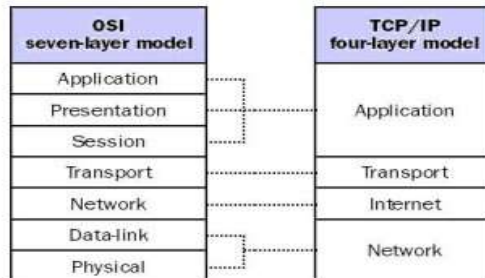
- Basically, HTTP is a TCP/IP based communication protocol, that is used to deliver data (HTML files, image files, query results, etc.) on the World Wide Web. The default port is TCP 80, but other ports can be used as well. It provides a standardized way for computers to communicate with each other. HTTP specification specifies how clients' request data will be constructed and sent to the server, and how the servers respond to these requests.
- **HTTP is connectionless:**
- **HTTP is media independent:**
- **HTTP is stateless**

Overview of HTTP

HTTP



- Web clients (web browsers) and web servers use HTTP (Hypertext Transfer Protocol) to communicate with each other
- More generally, HTTP is an application-layer protocol for distributed systems





Overview of HTTP

Connectionless vs. connection-oriented network communications

	Connectionless	Connection-oriented
CONNECTION	Does not need to establish a connection to send packets	Must establish a connection before sending packets
AUTHENTICATION	Not required to send packets	Required before sending packets
ROUTING	Packets can follow any route	All packets follow the same route
SEQUENCE	Packets can be sent and received in any order	Packets are sent and received sequentially
VERIFICATION	No verification of packet delivery	Packet delivery acknowledged when it is received
BANDWIDTH	Requires less bandwidth	Requires more bandwidth
CONGESTION	Possible congestion issues	Congestion not an issue
RELIABILITY	Less reliable	More reliable

Overview of HTTP



Stateless

- ⚙️ Server requests based on information with each request
- ⚙️ Does not rely on the earlier requests information
- ⚙️ Server does not hold requests information
- ⚙️ Different servers can different information at once



Stateful

- ⚙️ Based on internet protocol which requires a tight state
- ⚙️ Requests based on the information relayed with each request
- ⚙️ Information stored from earlier requests
- ⚙️ Same server must be used to process all requests



XENONSTACK

Overview of HTTP

HTTP Requests

Browser/Client → Server

```
GET /index.html HTTP/1.1
Host: www.example.com
```

Browser/Client ← Server

```
HTTP/1.1 200 OK
Date: Mon, 24 Sep 2018 22:38:34 GMT
Content-Type: text/html; charset=UTF-8
Content-Encoding: UTF-8
Content-Length: 138
Last-Modified: Wed, 10 Jan 2018 23:11:55 GMT
Server: Apache/2.4.34 (Unix) (Red-Hat/Linux)
ETag: "3f80f-1b6-3e1cb03b"
Accept-Ranges: bytes
Connection: close

<html>
<head>
<title>An Example Page</title>
</head>
<body>
Hello World, this is a very simple HTML document.
</body>
</html>
```

Header

Message body

Wikipedia Contributors: Wikipedia, The Free Encyclopedia, 16 September 2018 23:26
https://en.wikipedia.org/wiki/Hypertext_Transfer_Protocol. [accessed 13 Sep 2017]



HTTP Get vs Post

Get vs. Post

In case of Get request, only limited amount of data can be sent because data is sent in header.

Get request is not secured because data is exposed in URL bar.

Get request can be bookmarked.

Get request is idempotent. It means second request will be ignored until response of first request is delivered.

Get request is more efficient and used more than Post.

1

In case of post request, large amount of data can be sent because data is sent in body.

1

2

Post request is secured because data is not exposed in URL bar.

2

3

Post request cannot be bookmarked.

3

4

Post request is non-idempotent.

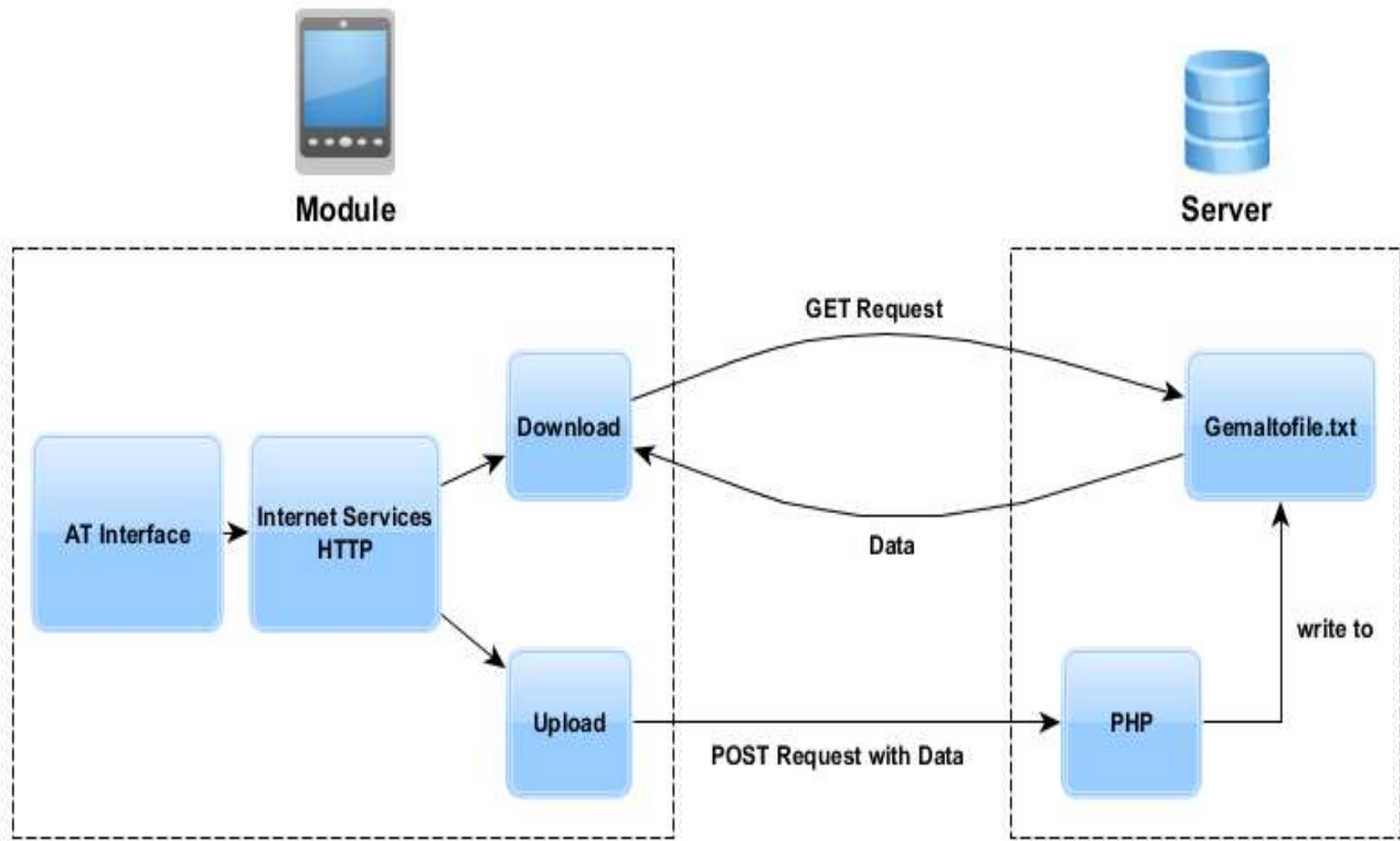
4

5

Post request is less efficient and used less than get.

5

HTTP Get vs Post





Server-side Programming

■ Uses

- Process user input.
- Compiles pages.
- Structure web applications.
- Interact with permanent storage (SQL, files).

■ Example Languages

- PHP
- Python
- ASP.Net in C#, C++, or Visual Basic.
- Nearly any language (C++, C#, Java). These were not designed specifically for the task but are now often used for application-level web services.



Client-side Programming

■ Uses

- Make interactive webpages.
- Make stuff happen dynamically on the web page.
- Interact with temporary storage, and local storage (Cookies, localStorage).
- Send requests to the server and retrieve data from it.
- Provide a remote service for client-side applications, such as software registration, content delivery, or remote multi-player gaming.

■ Example Languages

- JavaScript (primarily)
- HTML*
- CSS*
- Any language running on a client device that interacts with a remote service is a client-side language.



Simple vs. Multi-tiered Web Site Architecture

■ System architecture

- ❖ Arrangement of software, hardware, and tasks in an information system needed to achieve a specific functionality.

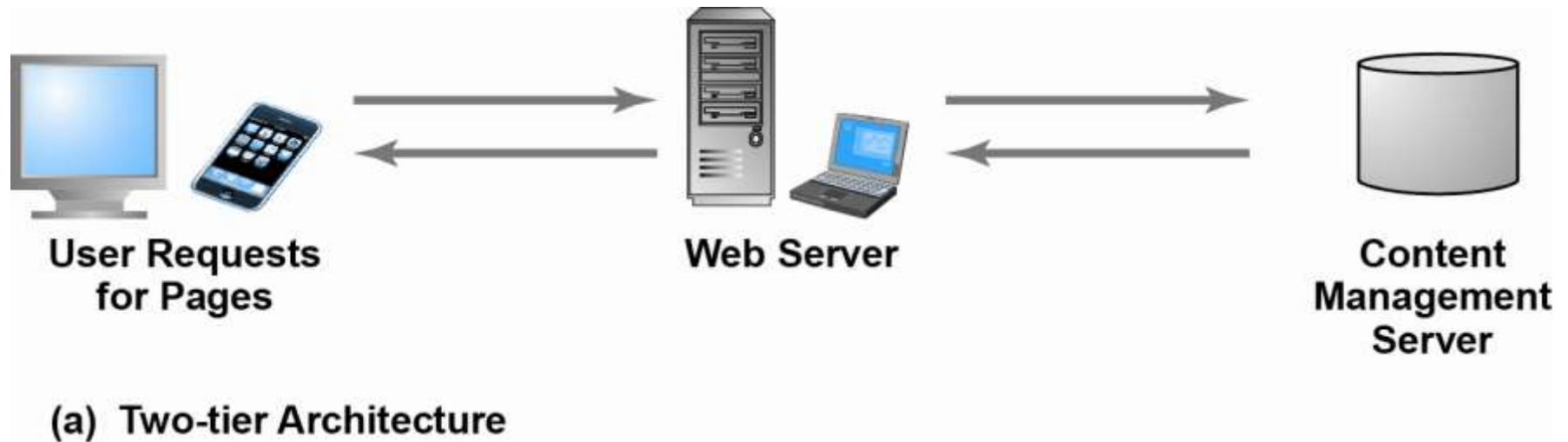
■ Two-tier

- ❖ Web server and database server

■ Multi-tier

- ❖ Web application and other servers
 - ❖ Backend, legacy databases
- 

Two-Tier E-commerce Architecture



Multi-Tier E-commerce Architecture



(b) Multi-tier Architecture

In a multi-tier architecture, a Web server is linked to a middle-tier layer that typically includes a series of application servers that perform specific tasks, as well as to a backend layer of existing corporate systems.



Web Server Software

■ Apache

- ❖ Leading Web server software (52% of market)
- ❖ Works with UNIX, Linux operating systems
- ❖ Comes loaded on IBM systems

■ Microsoft' s Internet Information Server (IIS)

- ❖ Second major Web server software (20% of market)
- ❖ Windows-based



Dynamic Page Generation Tools

■ Dynamic page generation:

- ❖ Page contents stored in database and fetched when needed
- ❖ Lowers menu cost
- ❖ Enables market segmentation

■ Common tools:

- ❖ Common Gateway Interface (CGI)
- ❖ Active server pages (ASP)
- ❖ Java Server Pages (JSP)
- ❖ Open Data Base Connectivity (ODBC), a std DB access method allows connections to any DB

■ Advantages

- ❖ Lowers menu costs
 - ❖ Permits easy online market segmentation
 - ❖ Enables cost-free price discrimination
 - ❖ Enables content management system (CMS)
- 



Application Servers

■ Web application servers:

- ❖ Is a server program in a computer in a distributed network that provides the business logic for an application program. Often viewed as a 3-tier application.
- ❖ It divides the application into
 - 1ST tier: front end – web browser GUI
 - 2nd middle tier: business logic
 - 3rd tier: back end – DB and transaction server



Distributed Systems

Distributed Systems: Fundamental Questions

Software developers have to consider a wide, but rather stable, range of **questions** including:

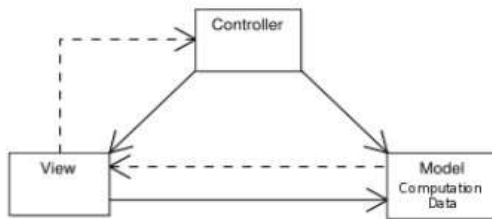
- Where can or should **computations** take place?
- Where can or should **data** be stored?
- How fast can data be transferred/**communicated**?
- What is the cost of data storage/computations/communication depending on how/where we do it?
- How robustly/securely can data storage/computations/communication be done depending on how/where we do it?
- How much energy is available to support data storage/computations/communication depending on how/where we do it?
- What is the legality of data storage/computations/communications depending on how/where we do it?

The **possible answers** to each of these questions is also rather stable, but the **'right' answers** change

Distributed Systems

Distributed Systems: Model-View-Controller

We use the **Model-View-Controller** software design pattern to discuss some of these questions in more detail:



- The **model** manages the **behaviour** and **data**
- The **view** renders the **model** into a form suitable for interaction
- The **controller** receives user input and translates it into instructions for the **model**

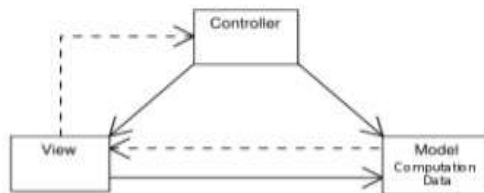
1 Where should the **view** be **rendered**?

- On the user's computer
- On a central server (farm) possibly shared by a multitude of users

Distributed Systems

Distributed Systems: Model-View-Controller

We use the **Model-View-Controller** software design pattern to discuss some of these questions in more detail:



- The **model** manages the **behaviour** and **data**
- The **view** renders the **model** into a form suitable for interaction
- The **controller** receives user input and translates it into instructions for the **model**

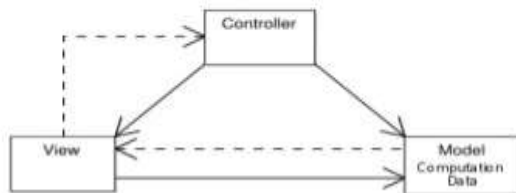
② Where should the **behaviour** of the **model** be computed?

- Close to the user,
on a single computer exclusively used by the user
- Away from the user,
on a central server (farm) shared by a multitude of users
- Distributed,
on several computers owned by a large group of users

Distributed Systems

Distributed Systems: Model-View-Controller

We use the **Model-View-Controller** software design pattern to discuss some of these questions in more detail:

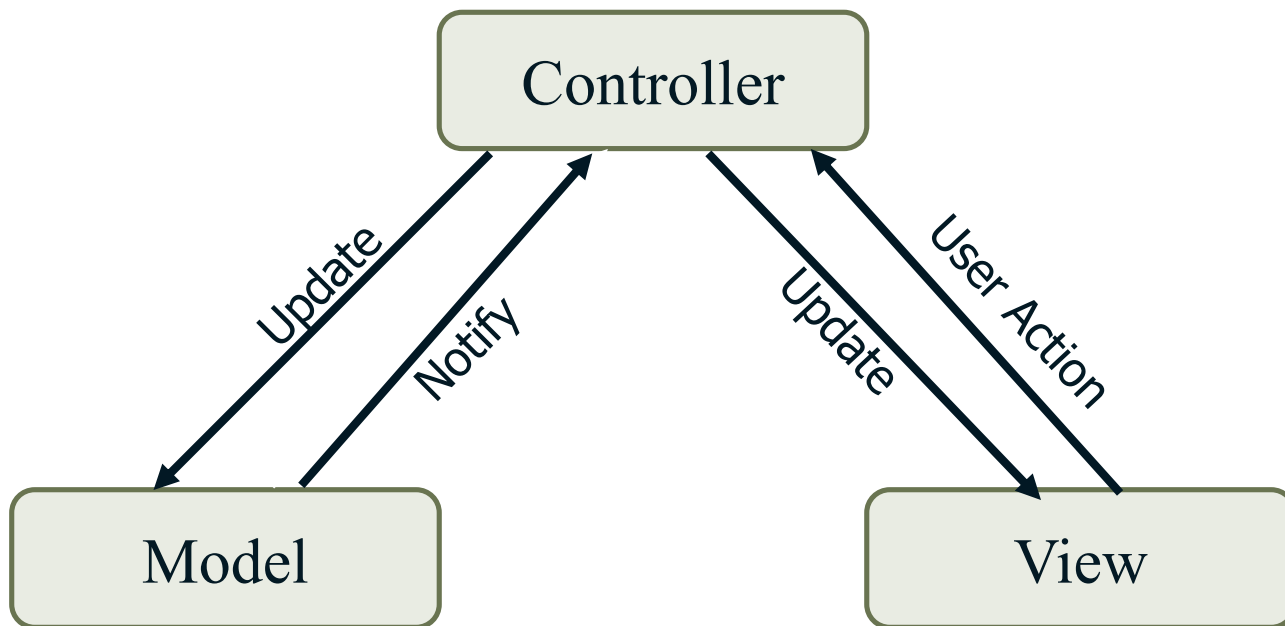


- The **model** manages the **behaviour** and **data**
- The **view** renders the **model** into a form suitable for interaction
- The **controller** receives user input and translates it into instructions for the **model**

③ Where should the **data** for the **model** be held?

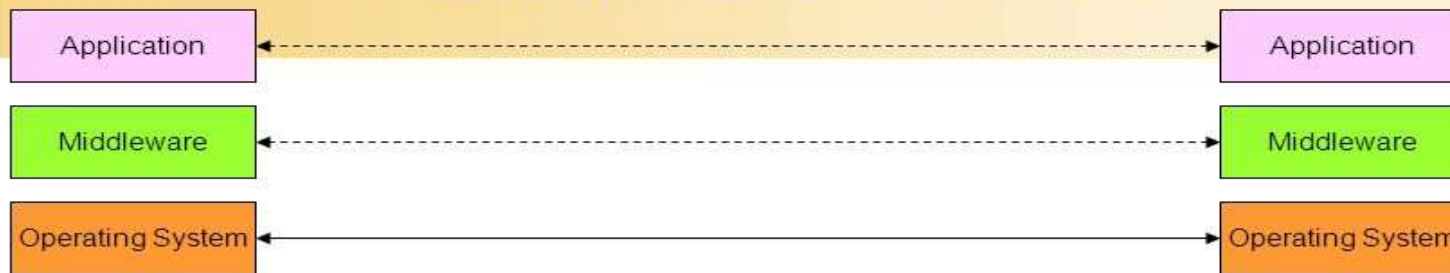
- Close to the user,
on a single computer exclusively used by the user
- Away from the user,
on a central server (farm) shared by a multitude of users
- Distributed,
on several computers owned by a large group of users

MVC Model



Middleware

What is Middleware?

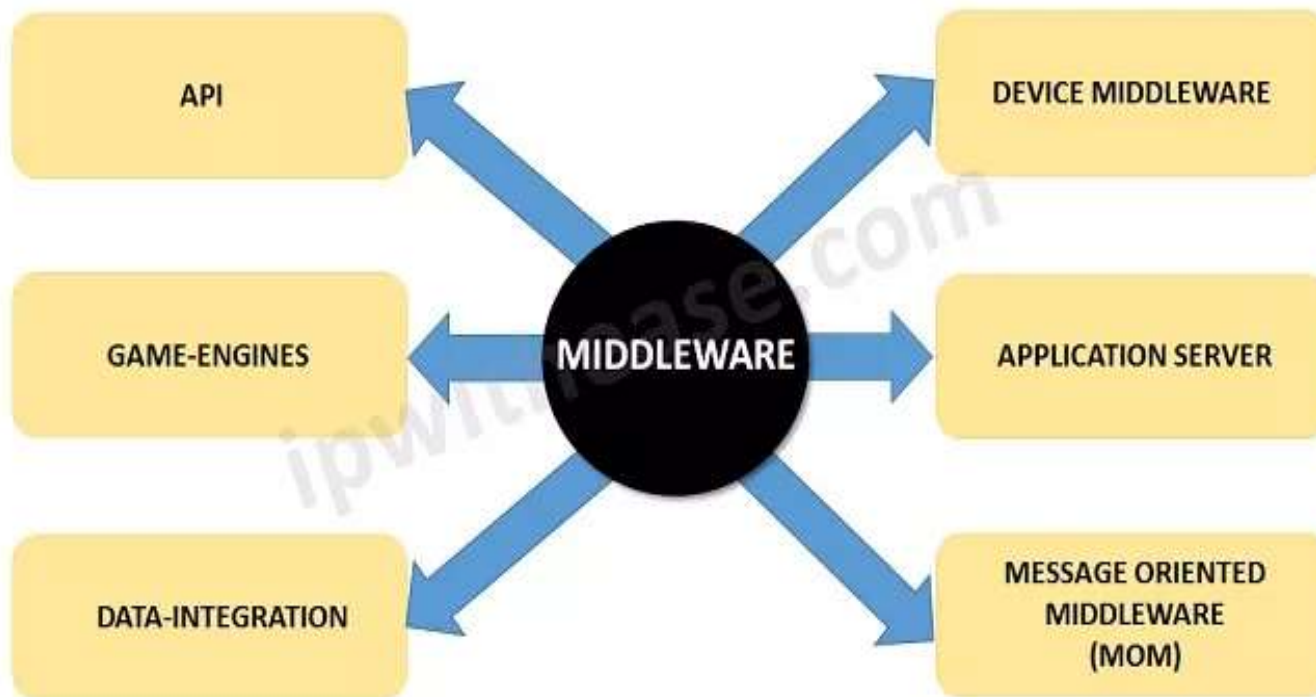


Software that functions as a conversion or translation layer.

- It is also a consolidator and integrator.
 - Custom-programmed middleware solutions have been developed for decades to enable one application to communicate with another that either runs on a different platform or comes from a different vendor or both.
- Today, there is a diverse group of products that offer packaged middleware solutions.

Middleware

Types Of Middleware

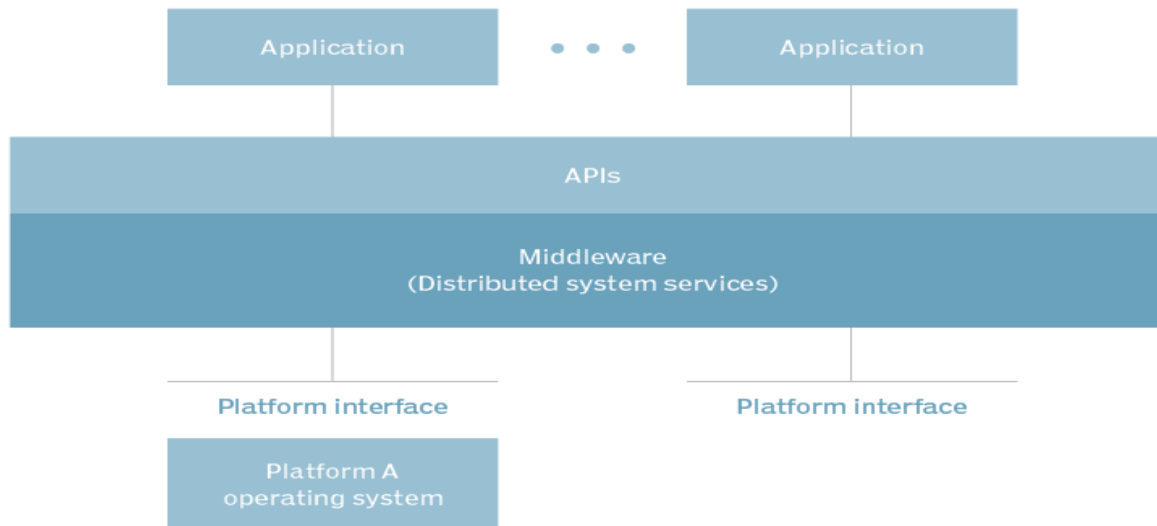


networkinterview.com

(An Initiative By ipwithease.com)

Middleware

Middleware architecture



Principles of Web Design

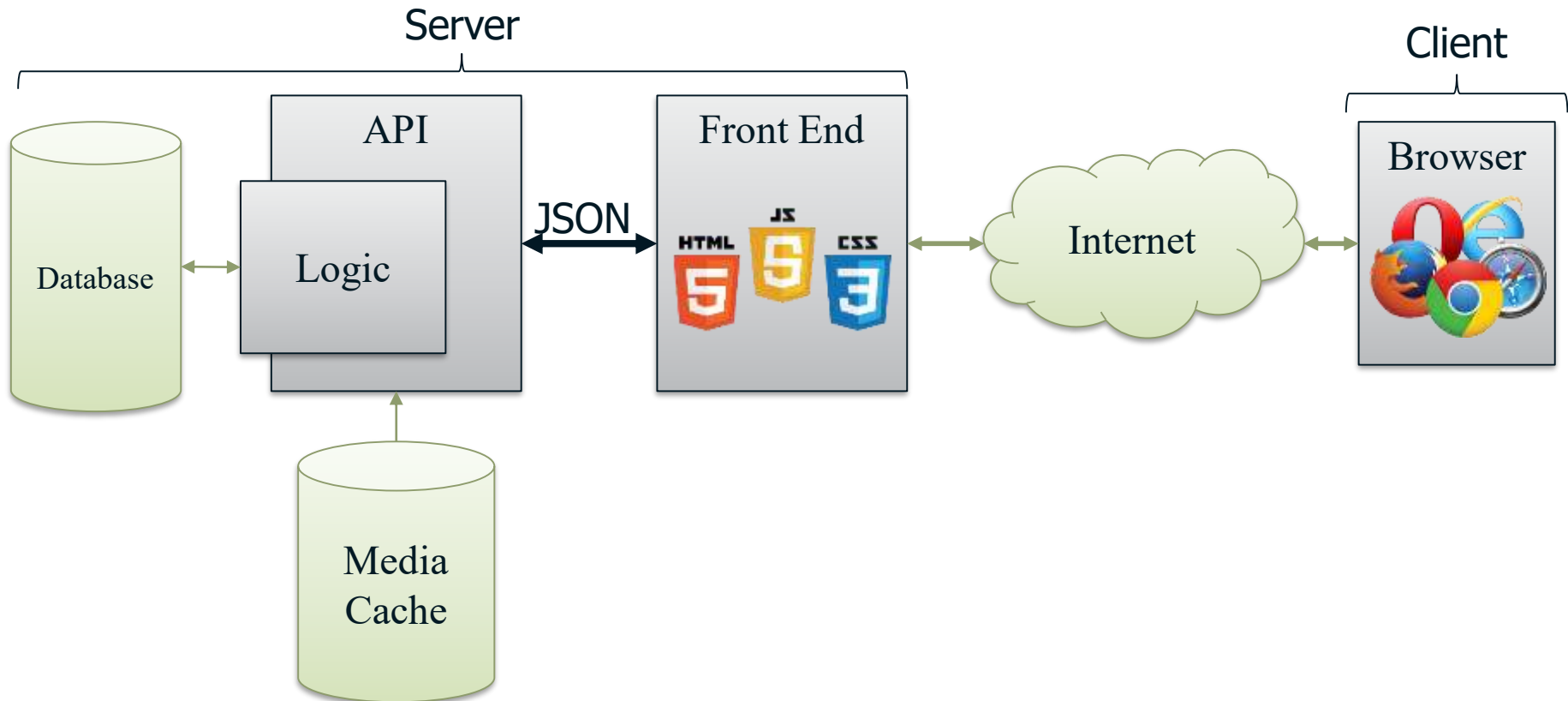
- Availability
- Performance
- Reliability
- Scalability
- Manageability
- Cost

Performance



- UI (Front End (DOM, Framework))
- Request Layer (Web API)
- Back End (Database, Logic)

Core Components of Web Applications



Front End Languages

- HTML/CSS
- Javascript
- Java (applets)

What is the most popular?

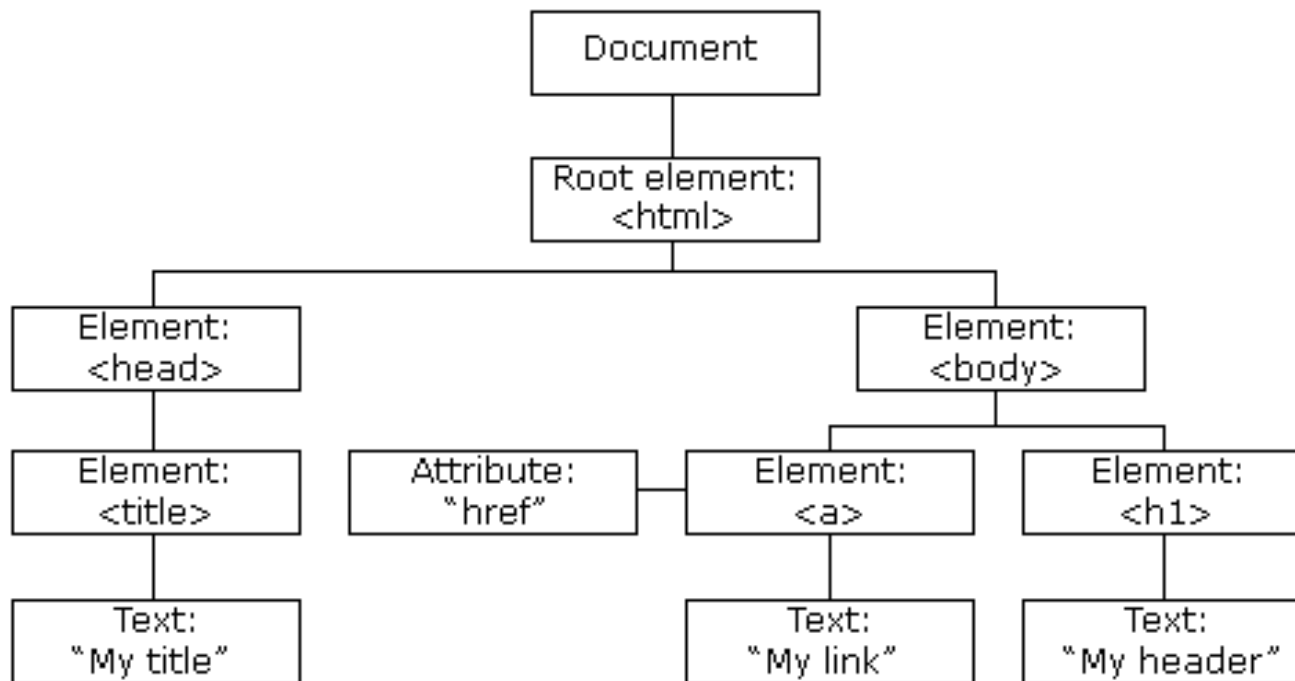
Answer: Javascript/HTML/CSS is the only real option for front-end native languages and is basically the standard. But there are many variations on JavaScript that are used.





DOM (Document Object Model)

- Document Object Model makes every addressable item in a web application an Object that can be manipulated for color, transparency, position, sound and behaviors.
- Every HTML Tag is a DOM object





What is a Framework?

- **Software Framework designed to reduce overhead in web development**
- **Types of Framework Architectures**
 - ❖ Model-View-Controller (MVC)
 - ❖ Push vs Pull Based
 - Most MVC Frameworks use push-based architecture “action based” (Django, Ruby on Rails, Symfony, Stripes)
 - Pull-based or “component based” (Lift, Angular2, React)
 - ❖ Three Tier Organization
 - Client (Usually the browser running HTML/Javascript/CSS)
 - Application (Running the Business Logic)
 - Database (Data Storage)
- **Types of Frameworks**
 - ❖ Server Side: Django, Ruby on Rails
 - ❖ Client Side: Angular, React, Vue

